



Singularity Mosaic

”

Welcome to Singularity Mosaic, a project dedicated to exploring the enigmatic universe. We'll analyze astronomical data, focusing on black holes. Our journey will cover the history of astronomy and modern coordinate systems. We'll also dive into fascinating deep-sky objects.

“

Event Horizon Telescope: Seeing the Unseeable



Historic Black Hole Image
M87* was imaged in 2019.



Global Telescope Network
The EHT uses Very Long Baseline
Interferometry (VLBI).



Unprecedented Resolution
Imaging an orange on the moon is
now possible.



Future Cosmic Targets
Next, imaging Sagittarius A* at the
Milky Way's center.

How EHT works:

Global Synchronization

The EHT's 9 telescopes are time-synced using hydrogen maser atomic clocks.

Central Data Correlation

Data is flown (literally!) to supercomputing centers (MIT, Max Planck) for processing.

Petabytes of Data

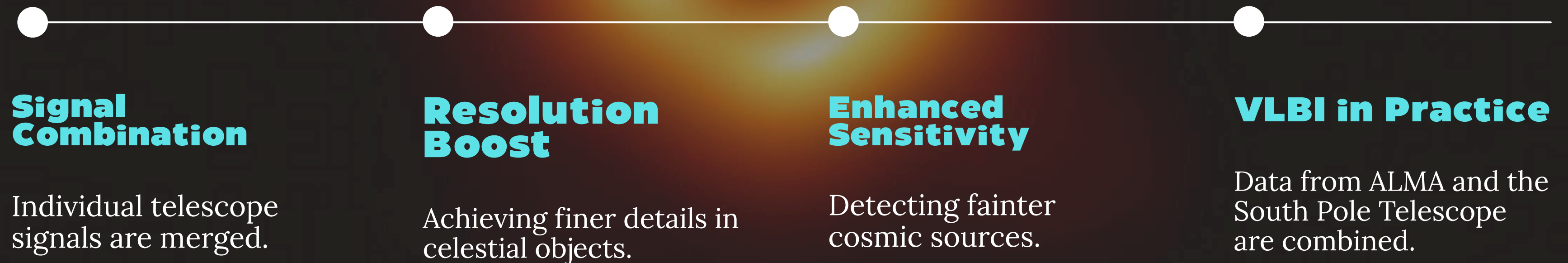
Each site records raw radio signals onto physical hard drives, up to 4 PB total.

Image Reconstruction

Powerful algorithms turn noisy, sparse data into a visible black hole

Interferometry: Virtual Telescopes

Interferometry combines signals from multiple telescopes to create a single, larger virtual telescope. This technique significantly improves resolution and sensitivity. Very Long Baseline Interferometry (VLBI) is a powerful application, combining distant observatories.



UV Coverage and Fourier Transforms

The UV Plane

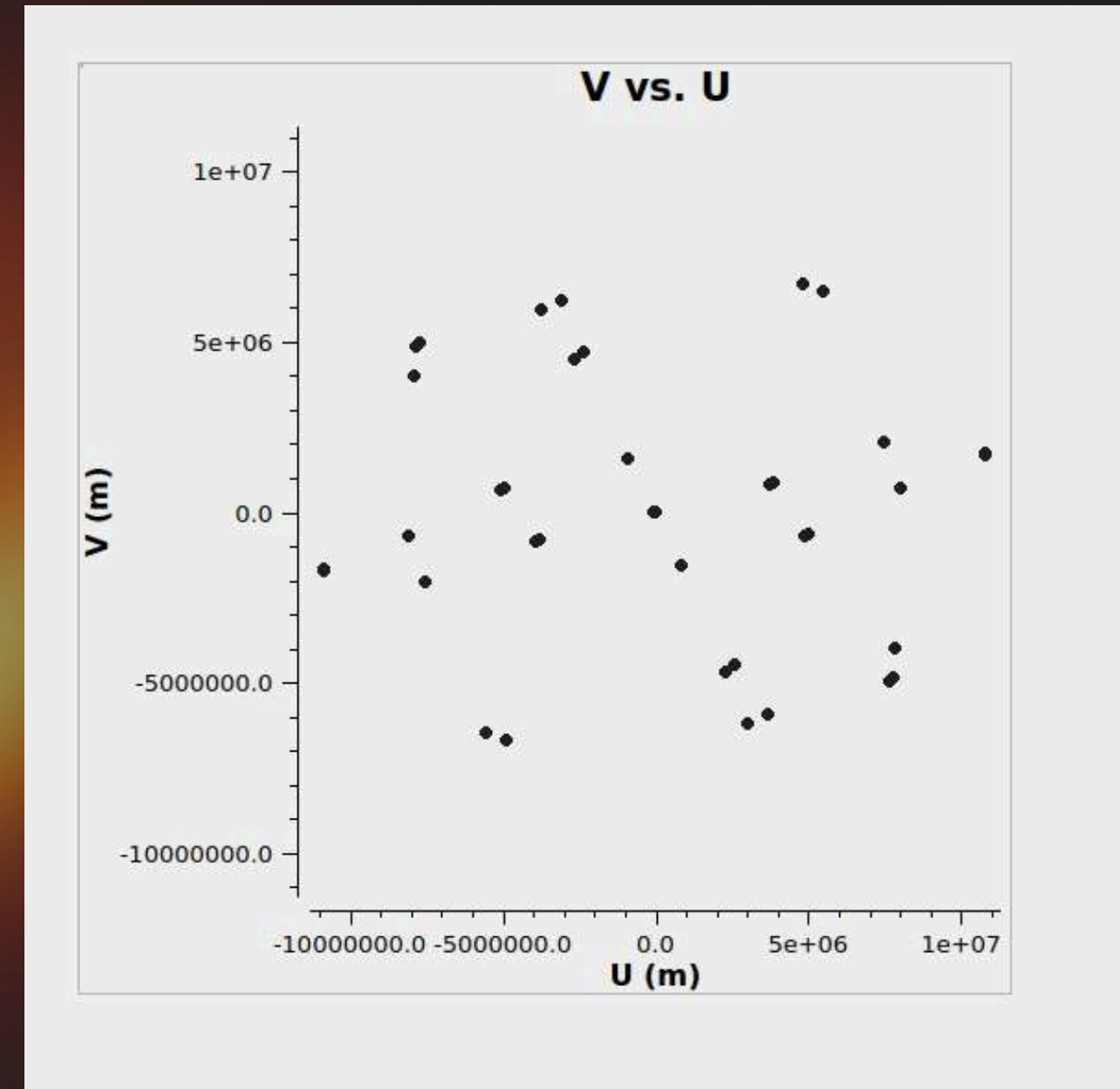
This plane represents the spatial frequencies sampled by an interferometer. Each baseline provides a point.

Inverse Fourier Transform

This mathematical tool converts raw UV data into a celestial image. It's the core of image reconstruction.

Incomplete Coverage Challenges

When UV coverage is sparse, advanced image reconstruction techniques are necessary.



From Frequency to Images

Spatial Frequencies

The Fourier Transform decomposes an image into its constituent spatial frequencies.

Convolution Theorem

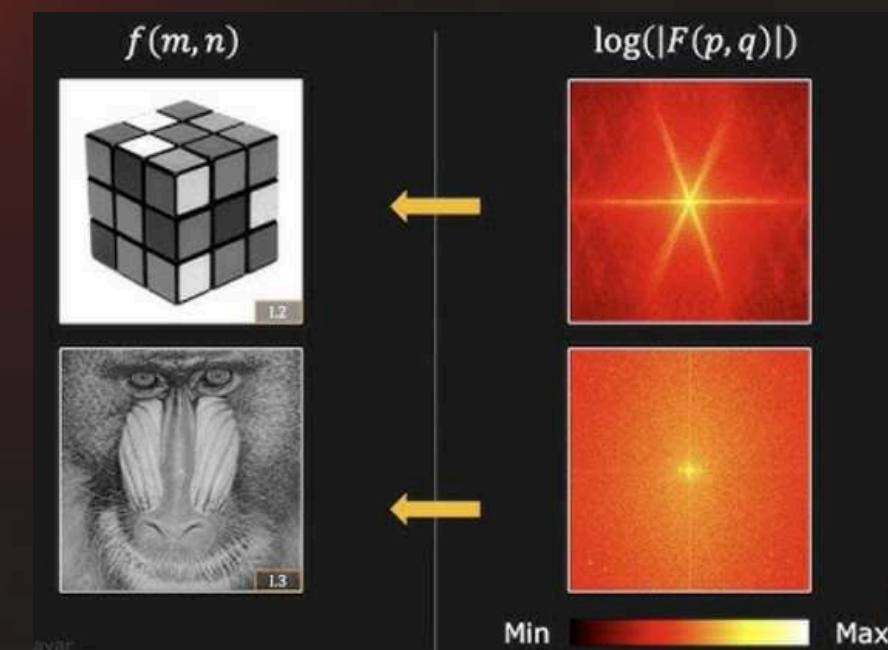
This theorem simplifies complex calculations in the frequency domain. The observed image is therefore a convolution of the true sky brightness with the point spread function (PSF) or synthesized beam.

Applications

Used in image filtering, compression, and reconstruction. MRI image formation is a prime example.

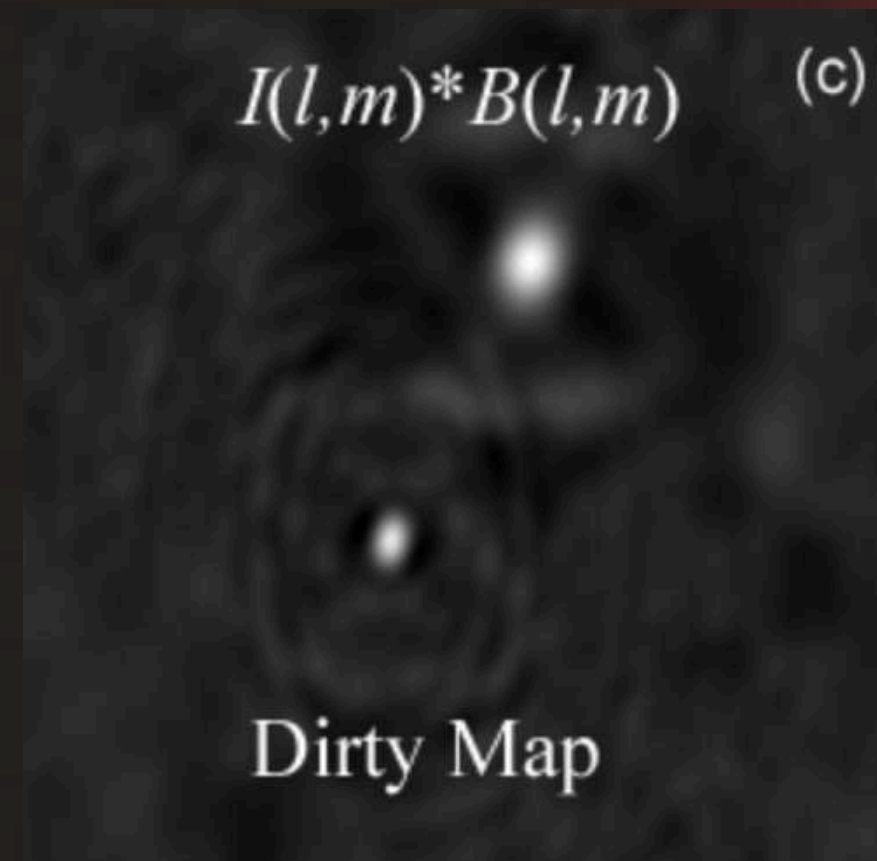
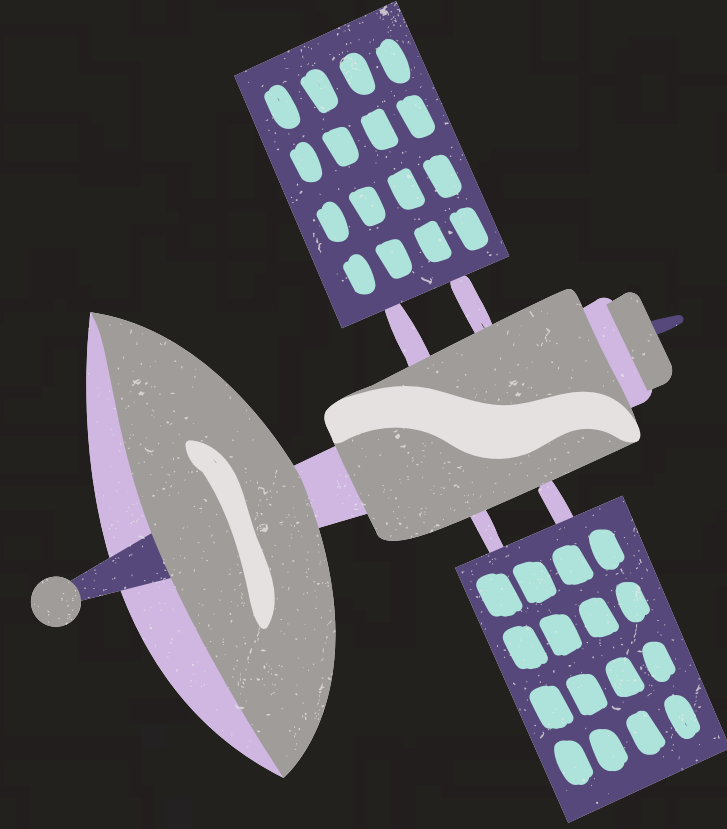
Amplitude & Phase

Both amplitude and phase components encode vital image information.



What is Dirty Image?

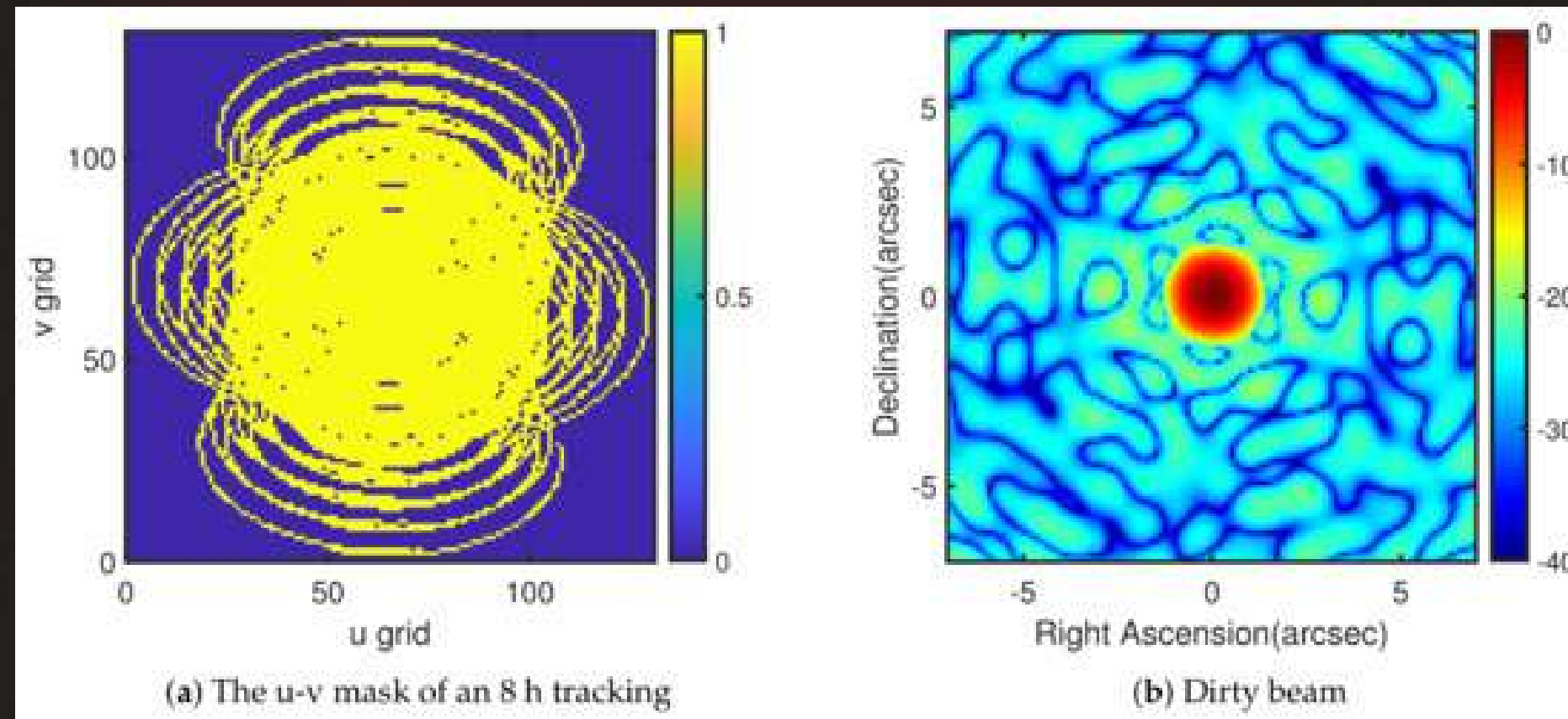
- The first image produced by a radio interferometer (array of radio telescopes).
- Created by taking the inverse Fourier transform of measured data ("visibilities").
- Not a true sky image; it contains distortions and artifacts.



Why it is Dirty?

- The image is a mix of the real sky and unwanted patterns (sidelobes, noise).
- These patterns come from the **Dirty Beam**.

What is Dirty Beam?

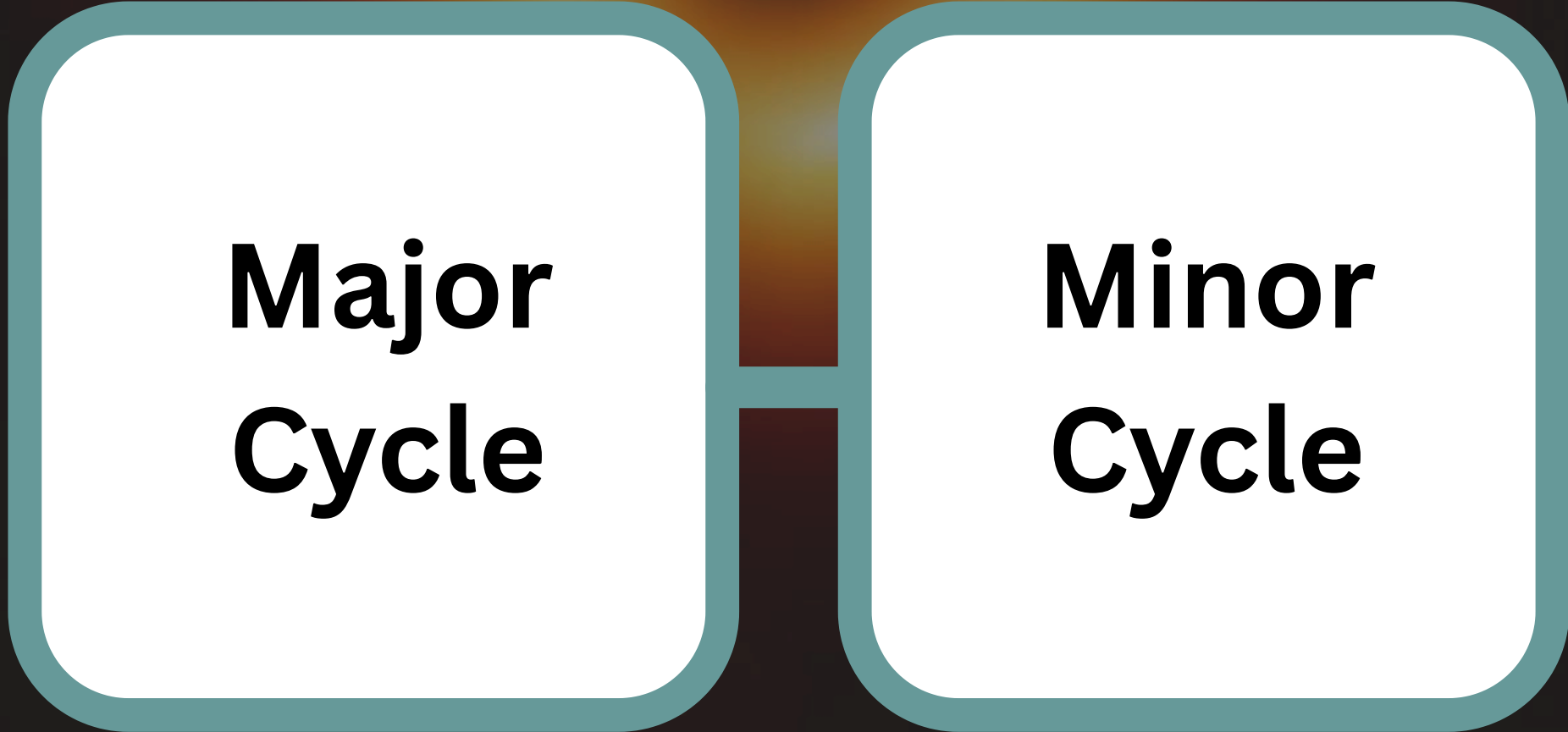


- The “point spread function” (PSF) of the interferometer array.
- Shows how a single point in the sky would appear in the dirty image.
- Has a sharp central peak but many sidelobes (ripples and negative regions).
- Caused by incomplete and uneven sampling of the sky by the telescope array.

CLEAN Algorithm: Refining the Image

CLEAN is a foundational algorithm in radio astronomy imaging, designed to remove artifacts caused by incomplete data from radio interferometers. By iteratively identifying and subtracting the effects of the instrument's response (the "dirty beam"), CLEAN enables astronomers to reconstruct much clearer and more accurate images of the radio sky.

Two Cycles for CLEAN Algorithm



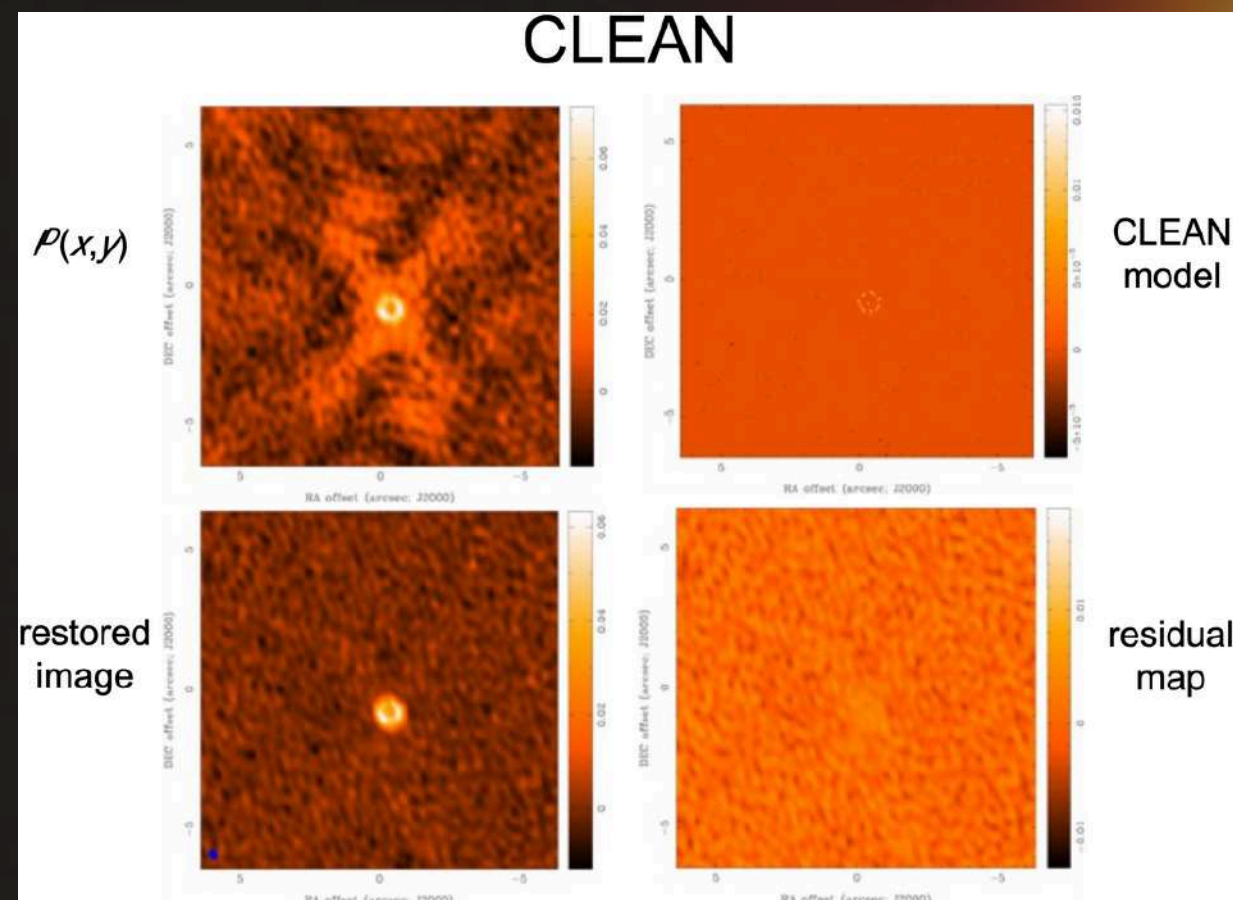
**Major
Cycle**

**Minor
Cycle**

How CLEAN Algorithm works

Find the Peak: Locate the brightest pixel in the current residual image

Record Component: Add a fraction (gain factor) of this peak to a clean component list



Subtract Beam: Remove a scaled version of the dirty beam pattern centered on the peak location

Iterate: Repeat until stopping criteria are met (threshold or maximum iterations)

Restore: Convolve clean components with an idealized "clean beam" and add residuals

Image Cleaning Process

Minor Cycle

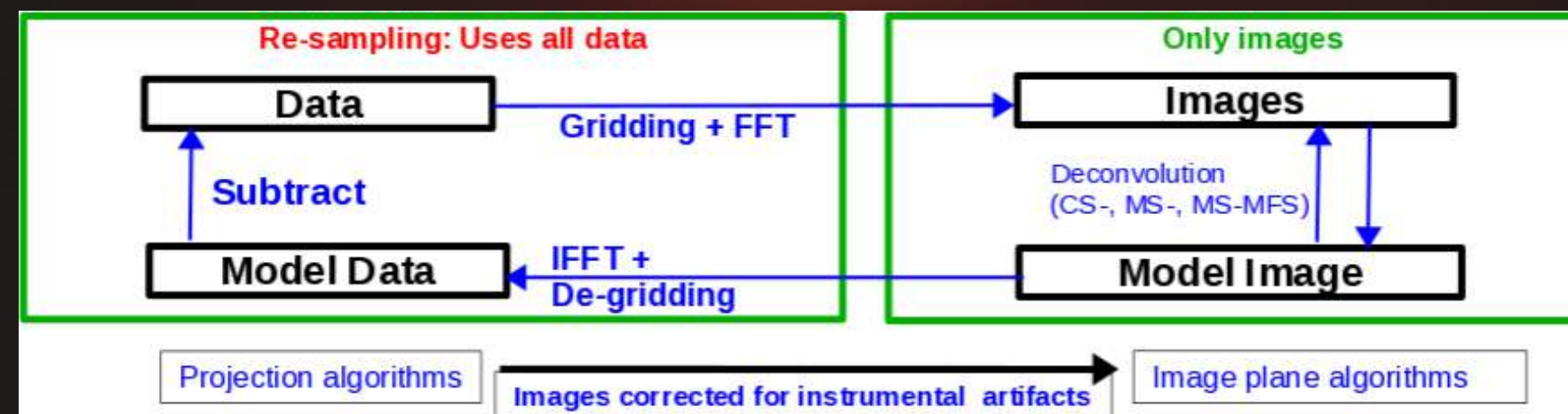
Image Domain Processing:

- CLEAN components are searched for and subtracted from the image
- Only the central portion of the dirty beam is used for subtraction
- Operates on a list of the largest residuals (not the full image)
- Much faster than using the full PSF for each subtraction
- The loss in accuracy is the trade-off price we pay to cut down the computational cost.

Major Cycle

Visibility Domain Correction:

- The list of CLEAN components from minor cycles are processed through Fast-Fourier-Transformation.
- Components are subtracted from the original visibility data.
- A new residual image is computed from updated visibilities.
- Corrects for errors that may have accumulated during minor cycle approximations.



CASA: Data Processing and Visualization

CASA Installation

Installing Common Astronomy Software Applications is the first step.

```

Time      Priority  Origin  Message
2025-06-12 18:37:29 INFO    :casa: The latest version is already installed in /Users/psrestvnu/.casa/data
2025-06-12 18:37:33 INFO    :casa: measure_update ... acquiring the lock ...
2025-06-12 18:37:35 INFO    :casa: lock file is not empty : /Users/psrestvnu/.casa/data/update.lock
2025-06-12 18:37:35 INFO    :casa: The lock file at /Users/psrestvnu/.casa/data is not empty.
2025-06-12 18:37:35 INFO    :casa: A previous attempt to update path may have failed or exited prematurely.
2025-06-12 18:37:35 INFO    :casa: Remove the lock file and set force to True with the desired version (default to most recent).
2025-06-12 18:37:35 INFO    :casa: It may be best to completely reconfigure path using pull_data and measure_update.
2025-06-12 18:37:35 INFO    :casa: lock file is not empty : /Users/psrestvnu/.casa/data/update.lock
2025-06-12 18:37:35 INFO    :casa: visit https://casadocs.readthedocs.io/en/stable/notebooks/external-data.html for more information
2025-06-12 18:37:35 INFO    :casa:
2025-06-12 18:37:35 INFO    :casa: Python version 3.12.4
2025-06-12 18:37:35 INFO    :casa: CASA Version CASAUTILS 6.7.0.31
2025-06-12 18:37:35 INFO    :casa: optional configuration file not found, continuing CASA startup without it
2025-06-12 18:37:35 INFO    :casa: loaded config file : /Applications/CASA.app/Contents/Frameworks/Python.framework/Versions/3.12/lib/python3.12/site-packages/casacore/config.py
2025-06-12 18:37:35 INFO    :casa:
2025-06-12 18:37:35 INFO    :casa: config values
2025-06-12 18:37:35 INFO    :casa: app = False
2025-06-12 18:37:35 INFO    :casa: cachedir = /Users/psrestvnu/.casa
2025-06-12 18:37:35 INFO    :casa: casacore_verbose = 1
2025-06-12 18:37:35 INFO    :casa: nlcars = "/>

```

Loading EHT Data

EHT data is loaded and calibrated within CASA for analysis.

Visualizing Black Holes

The software enables detailed visualization of black hole images and models.

Visualising black holes

1. The ‘importuvfits’ command converts raw UVFITS files into CASA’s Measurement Set (MS) format :

```
importuvfits(fitsfile="SR1_M87_2017_095_lo_hops_netcal_StokesI.uvfits", vis
="M87_095_lo.ms")
importuvfits(fitsfile="SR1_M87_2017_096_lo_hops_netcal_StokesI.uvfits", vis
="M87_096_lo.ms")
importuvfits(fitsfile="SR1_M87_2017_100_lo_hops_netcal_StokesI.uvfits", vis
="M87_100_lo.ms")
importuvfits(fitsfile="SR1_M87_2017_101_lo_hops_netcal_StokesI.uvfits", vis
="M87_101_lo.ms")
importuvfits(fitsfile="SR1_M87_2017_095_hi_hops_netcal_StokesI.uvfits", vis
="M87_095_hi.ms")
importuvfits(fitsfile="SR1_M87_2017_096_hi_hops_netcal_StokesI.uvfits", vis
="M87_096_hi.ms")
importuvfits(fitsfile="SR1_M87_2017_100_hi_hops_netcal_StokesI.uvfits", vis
="M87_100_hi.ms")
importuvfits(fitsfile="SR1_M87_2017_101_hi_hops_netcal_StokesI.uvfits", vis
="M87_101_hi.ms")
```

2. Use ‘listobs’ to examine the MS structure and metadata:

```
listobs(vis="M87_095_lo.ms")
```

3. The ‘plotms’ tool is used to visualise the UV-coverage, revealing the array geometry:

```
plotms(vis="M87_095_lo.ms", xaxis="u", yaxis="v", avgtime="1e8", avgscan=
True, coloraxis="spw")
```

4. ‘mstransform’ is used to generate corrected MS files, resolving pointing table inconsistencies:

```
mstransform(vis="M87_2017_095_lo.ms", outputvis="M87_2017_095_lo_fixed.ms
", datacolumn="data")
mstransform(vis="M87_2017_096_lo.ms", outputvis="M87_2017_096_lo_fixed.ms
", datacolumn="data")
mstransform(vis="M87_2017_100_lo.ms", outputvis="M87_2017_100_lo_fixed.ms
", datacolumn="data")
mstransform(vis="M87_2017_101_lo.ms", outputvis="M87_2017_101_lo_fixed.ms
", datacolumn="data")
```

5. Multiple MS files are merged using concat, preserving field names and time order:

```
concat(vis=vis=["M87_2017_095_lo_fixed.ms", "M87_2017_096_lo_fixed.ms", "
M87_2017_100_lo_fixed.ms", "M87_2017_101_lo_fixed.ms"],
concatvis="M87_combined_distinct.ms",
timesort=True, respectname=True, cospointing=False)
```

6. To plot the final image :

```
plotms(vis="M87_combined_distinct.ms", xaxis="u", yaxis="v",
coloraxis="field", avgchannel="9999", avgtime="9999")
```

Final image of M87 Black Hole

- Applying Tclean:

```
tclean(  
    vis='M87_combined_all.ms',  
    imagename='M87_final_image',  
    imsize=512,  
    cell='0.25uas',  
    weighting='uniform',  
    niter=100000,  
    threshold='0.5mJy',  
    cycleniter=500,  
    gain=0.1,  
    deconvolver='clark',  
    mask="",  
    restoringbeam=['20uas', '20uas', '0deg'], # Correct format  
    interactive=False,  
    usemask='user',  
    savemodel='modelcolumn'  
)
```

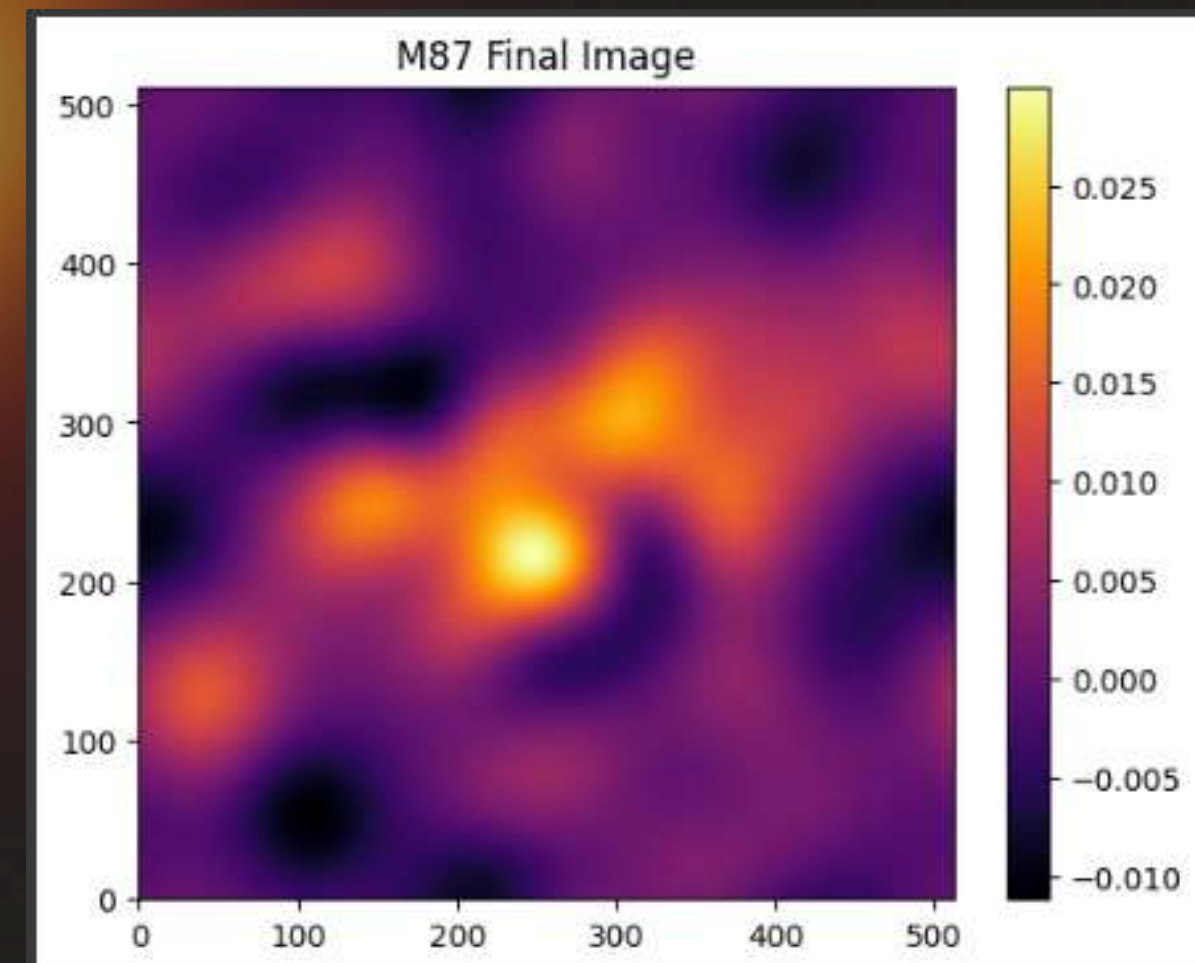
- exporting ms file to fits format:

```
exportfits(  
    imagename='M87_final_image.image',  
    fitsimage='M87_final_image.fits',  
    overwrite=True  
)
```

- Viewing the final image:

```
hdu = fits.open('M87_final_image.fits')[0]  
data = hdu.data.squeeze()  
  
plt.imshow(data, origin='lower', cmap='inferno')  
plt.colorbar()  
plt.title("M87 Final Image")  
plt.show()
```

Final Output of M87 Black hole:



Parameters of `tclean()`

Weighting: It introduces a weighting function that convolves with the original sampling function and changes the sampling function. To construct the image of black hole “uniform” weighting type is used because the final image has higher angular resolution.

Niter: It refers to max number of CLEAN iterations. Very low value will resemble the dirty image and very high value will do over-cleaning which may alter the apparent features of black hole.

Threshold: This sets the stopping point for CLEAN iterations. It does not affect the PSF but it definitely affects our final result. To get the value of threshold we use `imstat()` function on the residual image to get the RMS(Root mean square) and set the threshold $\sim 3 \times \text{RMS}$.

Loopgain: Controls how much detected peak is removed in each CLEAN iteration. It uses a float value between 0.0 and 1.0. By default its value is 0.1. Very high value of loopgain may result in over cleaning and very low value of loopgain may result in under cleaning.

Bayesian Statistics

Bayesian Statistics is a theory in the field of statistics based on the Bayesian interpretation of probability where probability expresses a degree of belief in an event. You start with a prior belief, collect data. You update your belief using the data this updated belief is called the posterior

Prior Probability $P(\theta)$

It is what you believe before seeing the data

Likelihood $P(D | \theta)$

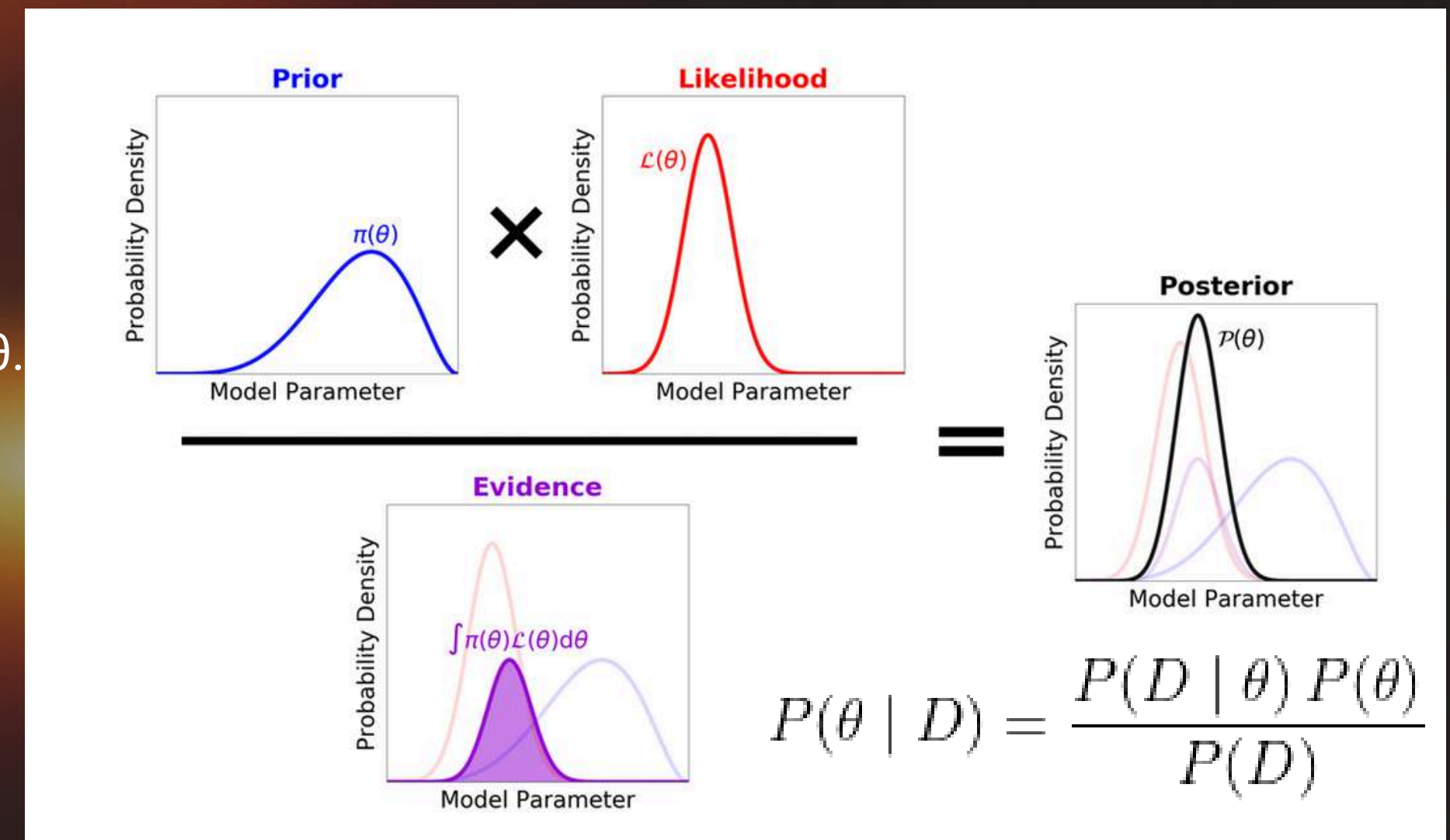
Probability of observing your data given the parameter θ .

Evidence $P(D)$

Overall probability of seeing the data .

Posterior Probability $P(\theta | D)$

What you believe about θ after seeing the data.



Sampling and Sampling Algorithms

Sampling simply means picking random values from a distribution. Most posterior distributions are too complex to compute analytically that's why we need sampling. A sampling algorithm is a method or set of steps to draw samples in a smart, practical way especially when direct sampling is hard.

Types of sampling algorithm

Simple Random Sampling: Pick samples completely at random.

Accept-Reject Sampling: Propose a sample, accept or reject based on probability.

MCMC (Markov Chain Monte Carlo): Build a chain of dependent samples that approximate the target distribution.

Metropolis-Hastings: A popular MCMC algorithm to handle complex posteriors.

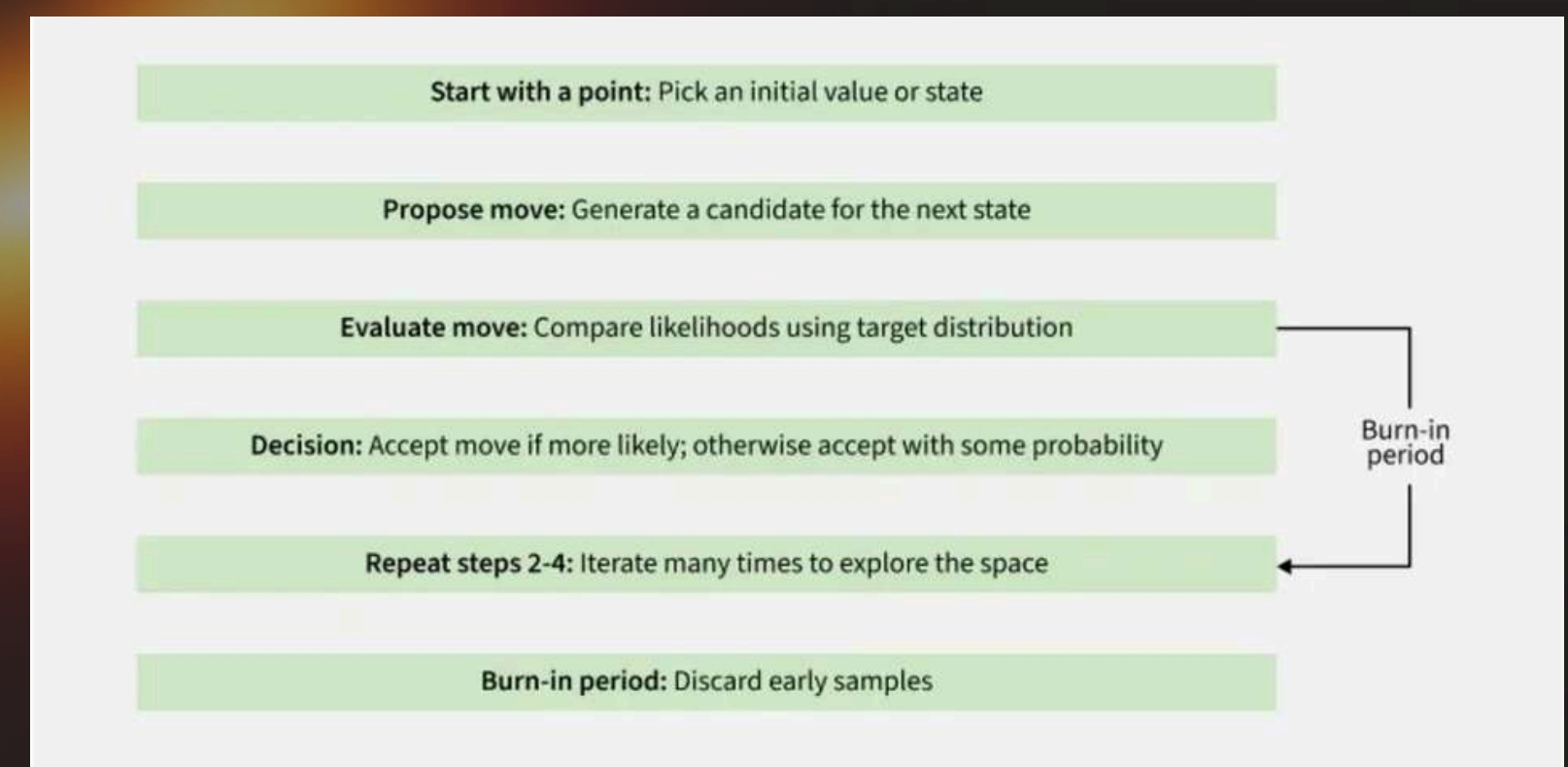
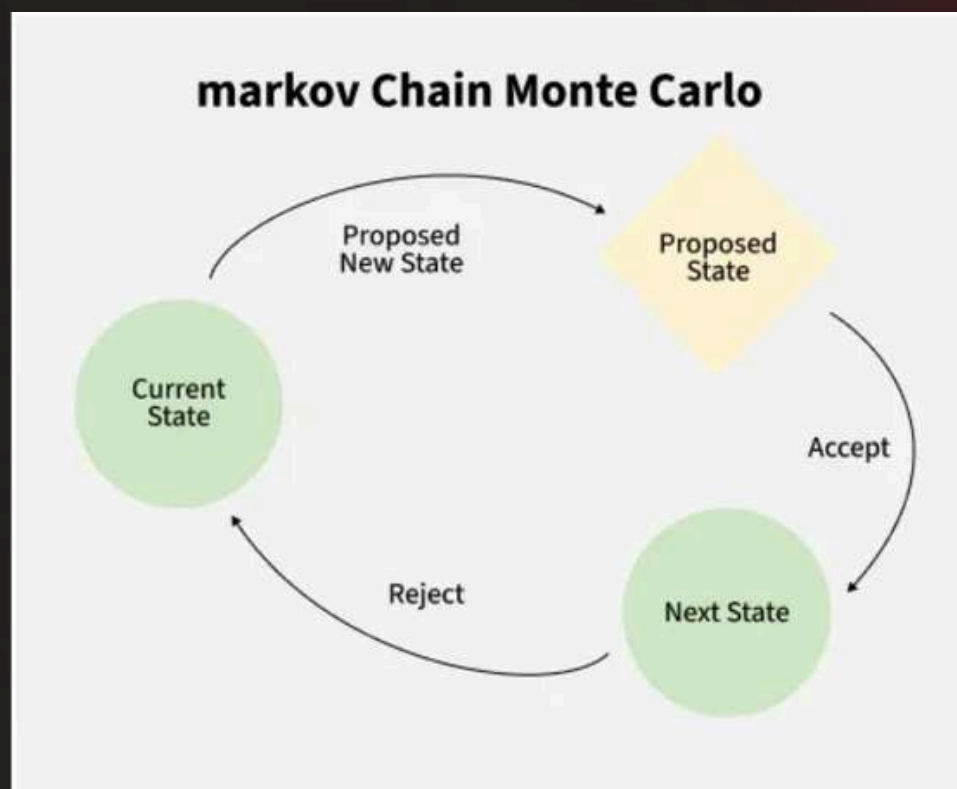
Markov chain Monte Carlo (MCMC)

MCMC is a sampling technique that builds a Markov chain to approximate complex posterior distributions. It avoids computing difficult integrals by generating samples that spend more time in high-probability regions, gradually converging to the true Bayesian posterior distribution.

$$P(\theta \mid \text{data}) \propto P(\text{data} \mid \theta) \times P(\theta)$$

$$P(\text{data}) = \int P(\text{data} \mid \theta) P(\theta) d\theta$$

How MCMC works:



MH - ALGORITHM

The Metropolis–Hastings algorithm is a Markov chain Monte Carlo (MCMC) method for obtaining a sequence of random samples from a probability distribution from which direct sampling is difficult. New samples are added to the sequence in two steps: first a new sample is proposed based on the previous sample, then the proposed sample is either added to the sequence or rejected depending on the value of the probability distribution at that point.

Algorithm 5.1: Metropolis-Hastings

Starting with $\mathbf{X}^{(0)} := (X_1^{(0)}, \dots, X_p^{(0)})$ iterate for $t = 1, 2, \dots$

1. Draw $\mathbf{X} \sim q(\cdot | \mathbf{X}^{(t-1)})$.

2. Compute

$$\alpha(\mathbf{X} | \mathbf{X}^{(t-1)}) = \min \left\{ 1, \frac{f(\mathbf{X}) \cdot q(\mathbf{X}^{(t-1)} | \mathbf{X})}{f(\mathbf{X}^{(t-1)}) \cdot q(\mathbf{X} | \mathbf{X}^{(t-1)})} \right\}.$$

3. With probability $\alpha(\mathbf{X} | \mathbf{X}^{(t-1)})$ set $\mathbf{X}^{(t)} = \mathbf{X}$, otherwise set $\mathbf{X}^{(t)} = \mathbf{X}^{(t-1)}$.

Evaluation of Image

- **Resolution**: The smallest angular scale at which two point sources can be distinguished.

For Optical Telescope :

$$\text{Resolving power} = \frac{1}{\Delta\theta} = \frac{d}{1.22 \lambda}$$

For Radio interferometer :

$$\text{Resolving power} = \frac{1}{\Delta\theta} = \frac{d}{\lambda}$$

- **Dynamic Range**: The ratio between the brightest signal and background noise level.

```
imstat(imageName='image.fits', region='off-source')
```

- **Noise** : Undesired fluctuations due to random errors.

$$\text{Dynamic Range} = \frac{I_{\max}}{\text{RMS}}$$

Enhancement of Image

Better Calibration:

We aim to improve gain calibration by solving for both amplitude (g) and phase (ϕ) errors:

$$V_{obs}(u, v) = g_i g_j^* V_{true}(u, v)$$

where $g_i = |g_i|e^{i\phi_i}$ is the complex gain for antenna i .

Weighting Schemes:

- **Natural weighting:** More sensitive, but lower resolution.
- **Uniform weighting:** Equalizes sampling across the (u,v) plane $\rightarrow$ higher resolution, noisier.
- **Robust weighting:** Interpolates between the two using a parameter R :

$$w_i = \frac{1}{1 + (R \cdot d_i)^2}$$

where d_i is the density of visibilities near a point in the (u,v) plane.

Deconvolution Methods:

We use deconvolution to recover the true sky brightness distribution $I(l,m)$ from the dirty image I_D , which is:

$$I_D(l, m) = I(l, m) * B(l, m)$$

where B is the dirty beam, the Fourier transform of the incomplete (u,v) coverage (the sampling function).

Post-Processing:

- **Gaussian smoothing:** Convolve the image with a Gaussian kernel to reduce high-frequency noise.
- **Median filtering:** Non-linear smoothing that preserves edges but reduces pixel-level noise.

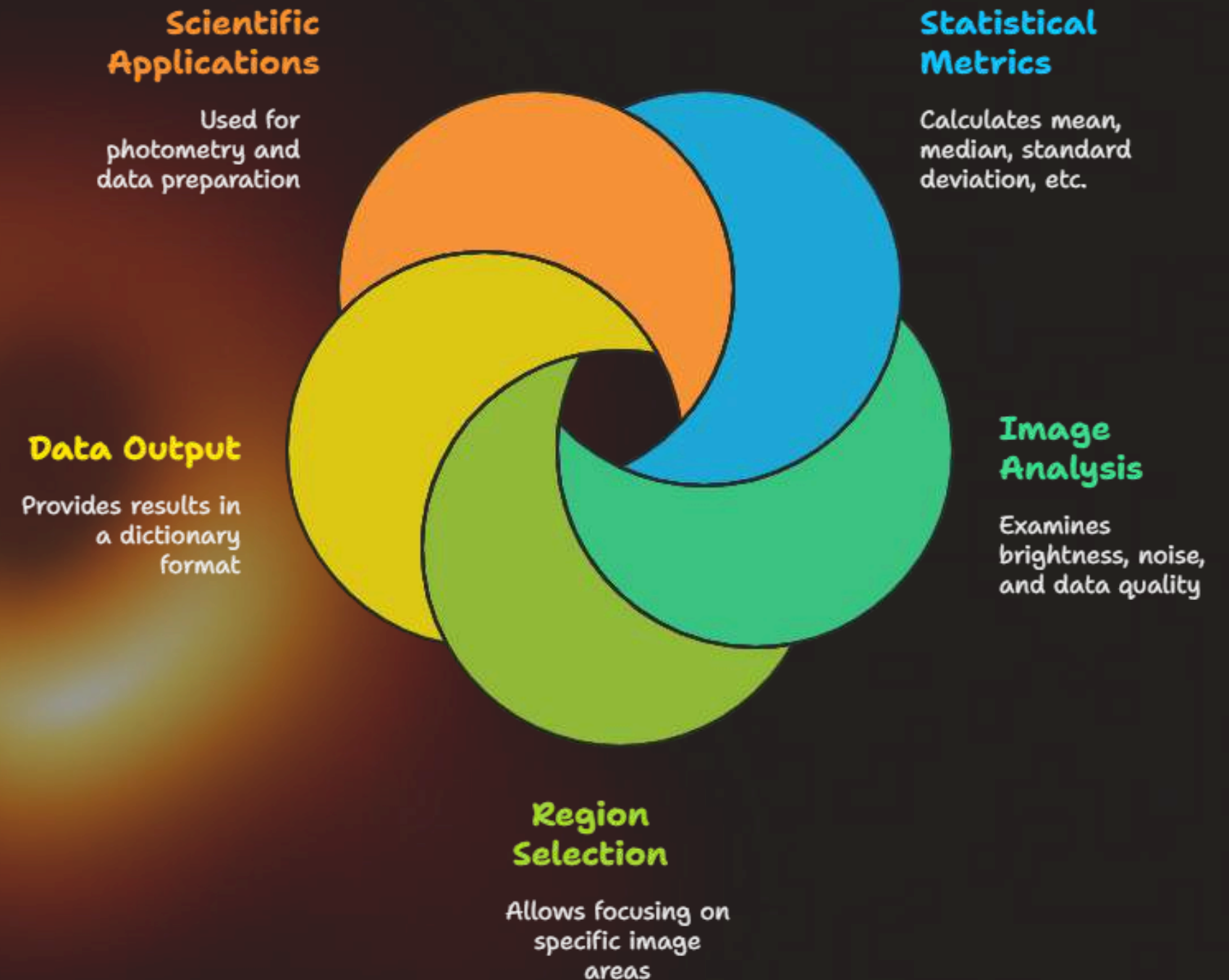
The imstat() Function

The imstat() function is a powerful tool in CASA (Common Astronomy Software Applications) that calculates and displays statistical information from astronomical images

The imstat() function answers questions like:

- How bright are the pixels in my image?
- What's the noise level?
- Where are the brightest and dimmest parts?
- What's the overall quality of my data?

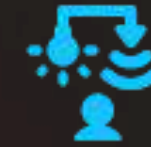
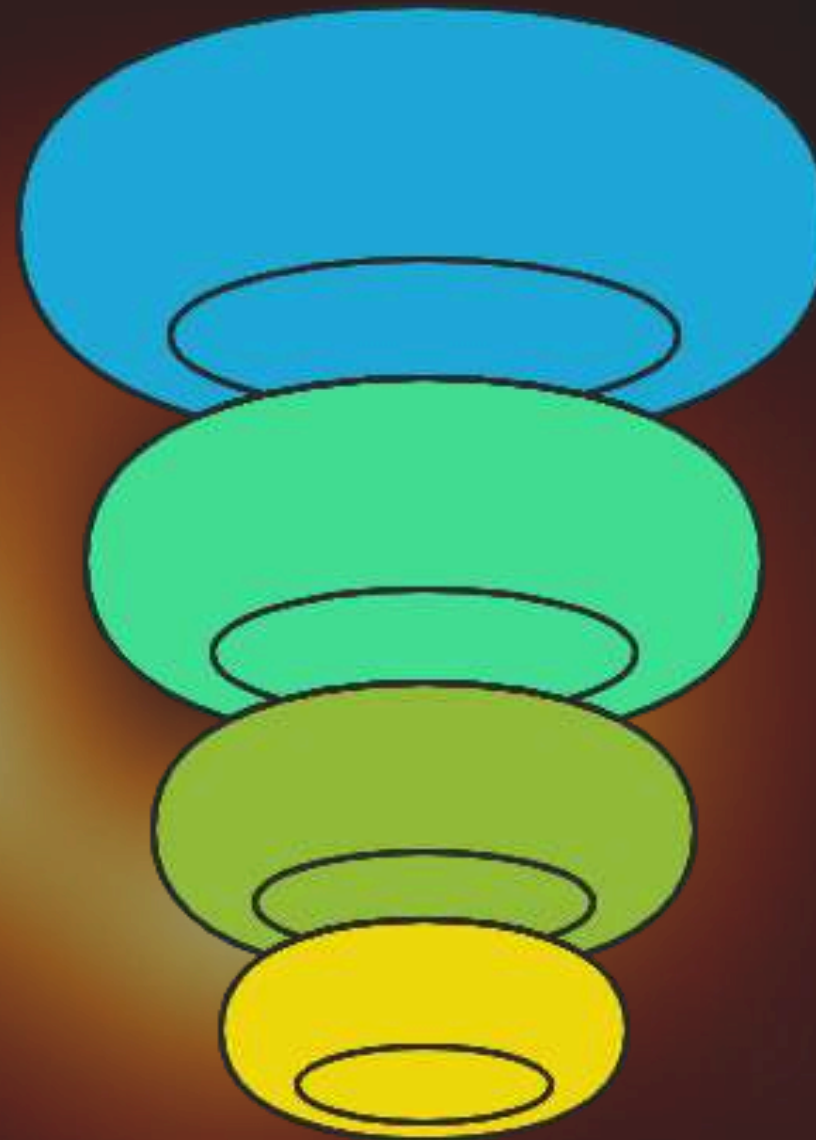
Exploring the imstat() Function



How Space Images Are Made

The images of galaxies or star clouds are made from numbers collected by special tools on telescopes.

These numbers tell us how much light is coming from different parts of the sky. Scientists then turn that light data into a picture that we can see and understand.



Data Collection

Sensors gather light data from space



Data Processing

Data is cleaned and combined for clarity



Color Assignment

Invisible light is assigned visible colors



Image Enhancement

Details are highlighted using mathematical tools

log_prior() function

```
def log_prior(image, l1w_weight=1.0, tv_weight=0.1):  
    l1_term = -l1w_weight * cp.sum(cp.abs(image))  
    tv_term = 0.0  
    if tv_weight > 0:  
        h_diff = cp.diff(image, axis=1)  
        v_diff = cp.diff(image, axis=0)  
        tv_term = -tv_weight * (cp.sum(cp.abs(h_diff)) + cp.sum(cp.abs(v_diff)))  
    return l1_term + tv_term
```

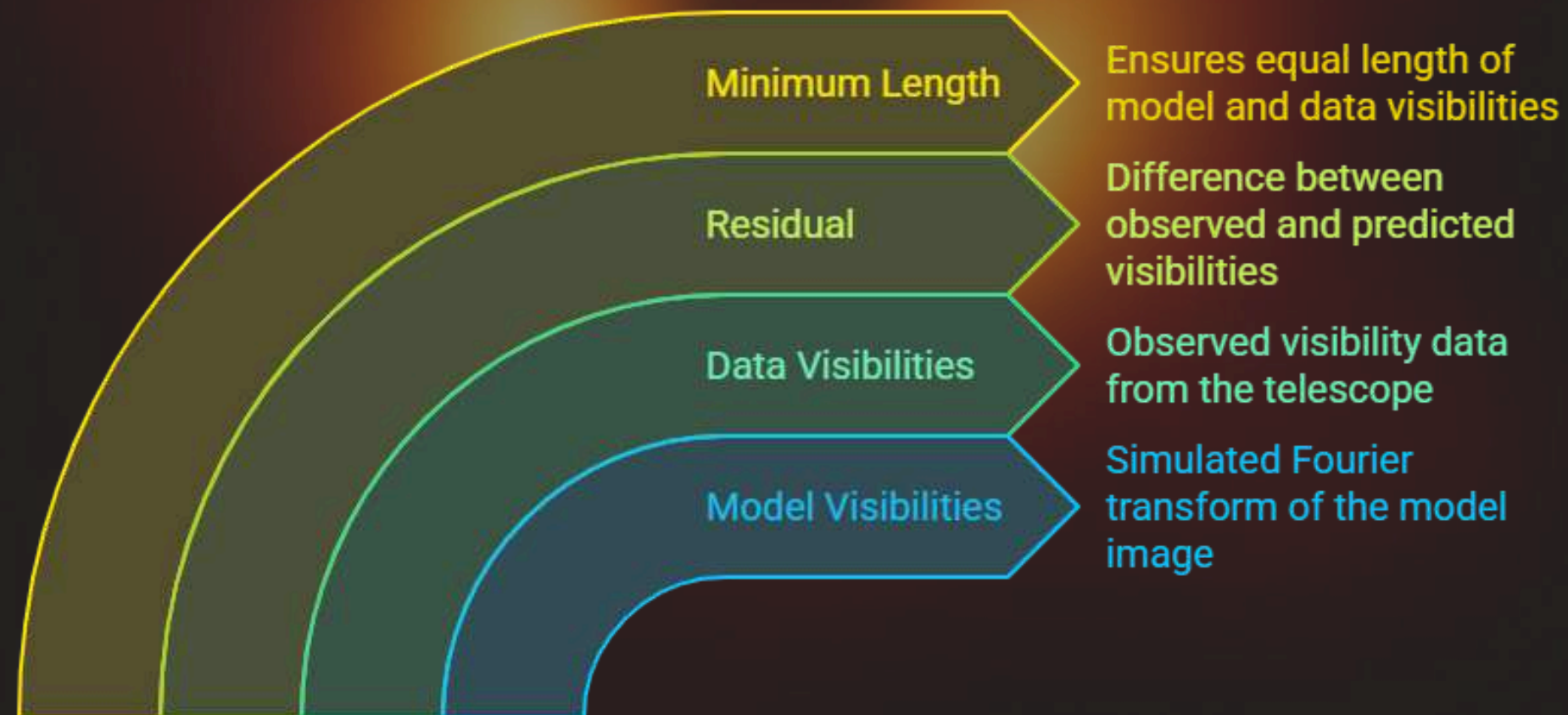
Components of log_prior()



log_likelihood() function

```
def log_likelihood(image, data_vis, u_coords, v_coords, pixel_scale, noise_std, batch_size=1000):  
    model_vis = image_to_visibilities_batch(image, u_coords, v_coords, pixel_scale, batch_size)  
    min_len = min(len(data_vis), len(model_vis))  
    data_vis = data_vis[:min_len]  
    model_vis = model_vis[:min_len]  
    residual = data_vis - model_vis  
    chi2 = cp.sum(cp.abs(residual)**2) / noise_std**2  
    return -0.5 * chi2
```

Components of Log-Likelihood Calculation



metropolis_sampler() function

```
def metropolis_sampler(init_image, data_vis, u_coords, v_coords, pixel_scale,
                      noise_std, n_samples=100, proposal_std=0.005,
                      llw_weight=0.1, tv_weight=0.01, burn_in=20, batch_size=1000):
    samples = []
    current = init_image.copy()
    current_logp = log_likelihood(current, data_vis, u_coords, v_coords, pixel_scale, noise_std, batch_size) + log_prior(current, llw_weight, tv_weight)
    n_accepted = 0

    for i in range(n_samples + burn_in):
        proposal = current + cp.random.normal(0, proposal_std, current.shape)
        proposal = cp.maximum(proposal, 0)
        proposal_logp = log_likelihood(proposal, data_vis, u_coords, v_coords, pixel_scale, noise_std, batch_size) + log_prior(proposal, llw_weight, tv_weight)
        log_alpha = proposal_logp - current_logp
        alpha = min(1.0, cp.exp(log_alpha).item())

        if cp.random.rand() < alpha:
            current = proposal
            current_logp = proposal_logp
            n_accepted += 1

        if i >= burn_in:
            samples.append(current.copy())

        if (i + 1) % 20 == 0:
            print(f"Step {i+1}/{n_samples+burn_in}, Acceptance rate: {n_accepted / (i + 1):.3f}")

    return cp.stack(samples)
```

Metropolis-Hastings MCMC Process

Initialize Metropolis Sampler

Sets up the MCMC algorithm

Generate Proposal Image

Creates a new candidate image

Calculate Log-Posterior

Computes the log-posterior for the proposal

Calculate Acceptance Ratio

Determines the acceptance probability

Apply Metropolis Rule

Decides whether to accept the proposal

Record Samples

Stores accepted samples after burn-in

Return Samples

Outputs the collected samples



Characterizing the Uncertainty!

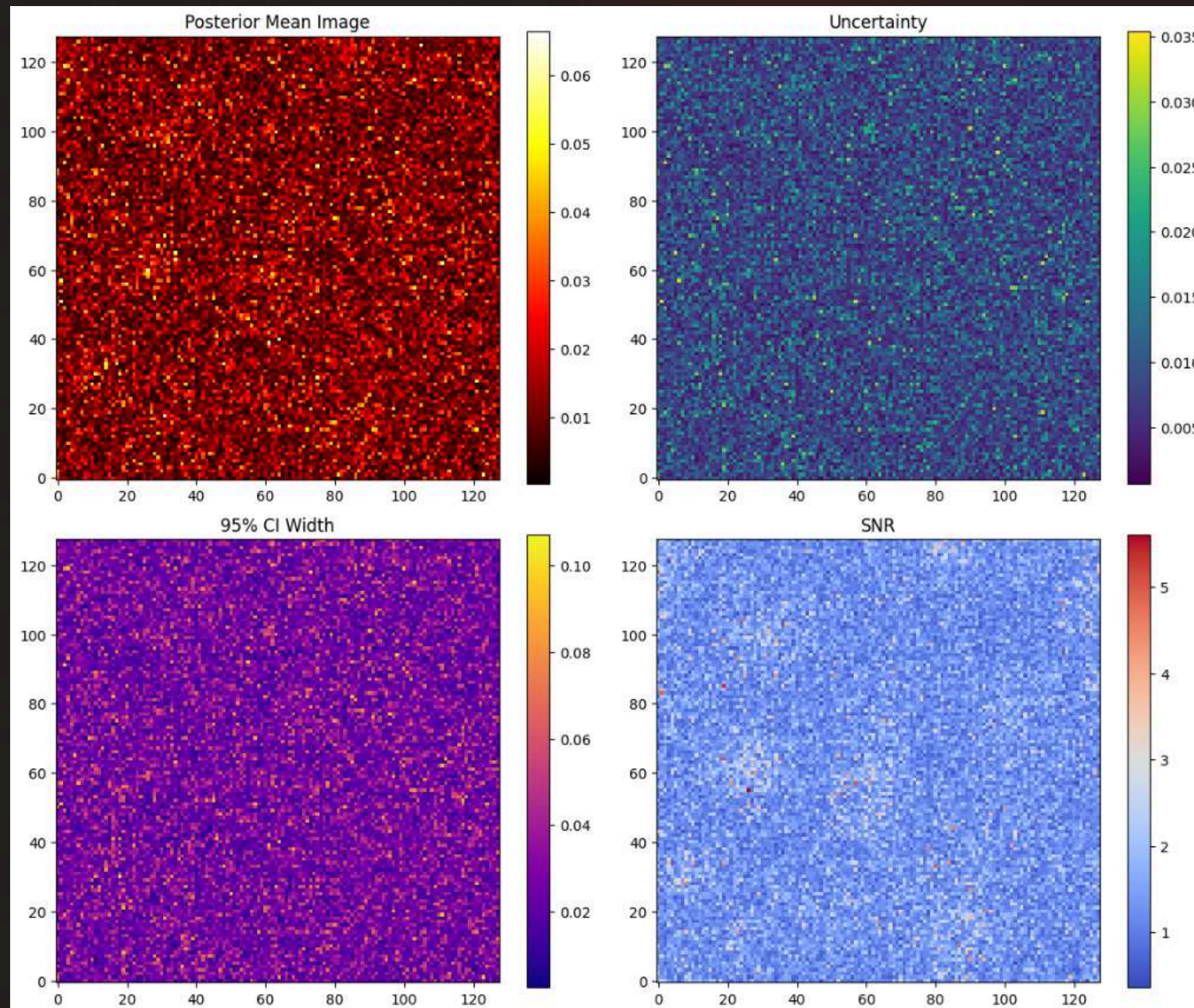


Image Quality & Uncertainty Metrics

- **Posterior mean image** confirms the central shadow and surrounding emission structure.
- **Uncertainty maps** and credible intervals help understand where the image is reliable.
- The **SNR map** shows that the core region of the image (the black hole shadow) is statistically robust.

THANK YOU